Parallel Implementation of Graph Neural Network Aggregation and Update Steps

A Note on Parallel GNN Computation

December 11, 2025

1 Introduction

This document outlines the parallel implementation strategy for the message passing mechanism in a Graph Neural Network (GNN), specifically detailing the aggregation and update steps suitable for a GPU-based execution model.

2 Core Steps of the GNN Layer

A single layer of the message-passing Graph Neural Network (GNN) consists of two primary sequential operations:

1. **Aggregate:** The feature vectors (messages) of the neighboring nodes are combined to form a neighborhood message vector.
2. **Update:** The node's current feature vector (message) is updated by incorporating the aggregated neighborhood message.

3 Detailed Pseudocode

The overall training and inference process for the GNN proceeds over a set number of iterations T, corresponding to the number of GNN layers.

1. **Initialization:** Initialize the messages (feature vectors) $\mathbf{h}_u^{(0)}$ of each node u to $\mathbf{0}$ or to the node's initial feature set.
2. **Iteration:** Repeat steps 3 and 4 for each node u for T iterations ($t = 1$ to T).
3. **Aggregation (Message Passing):**
   - Aggregate the messages of neighbor nodes $v \in \mathcal{N}(u)$ of node u.
   - This aggregation is performed using a function, often an averaging operation, which can be seen as an MLP followed by an activation function (if applied to the features *before* summing). The result is the neighborhood message vector $\mathbf{m}_{\mathcal{N}(u)}$:

$$\mathbf{m}_{\mathcal{N}(u)}^{(t)} = \frac{\sum_{v \in \mathcal{N}(u)} \mathbf{h}_v^{(t-1)}}{|\mathcal{N}(u)|}$$

4. **Update:**
   - Update the message (feature vector) of a node u from its own current message $\mathbf{h}_u^{(t-1)}$ and the aggregated message $\mathbf{m}_{\mathcal{N}(u)}^{(t)}$.
   - This update uses a combination of an MLP and an activation function. In a common setup, this involves a base update followed by a final concatenation:

Base Update (MLP + Activation):

$$\text{UPDATE}_{\text{base}}(\mathbf{h}_u, \mathbf{m}_{\mathcal{N}(u)}) = \sigma\left(\mathbf{W}_{\text{self}}\mathbf{h}_u^{(t-1)} + \mathbf{W}_{\text{neigh}}\mathbf{m}_{\mathcal{N}(u)}^{(t)}\right)$$

Final Update (Concatenation):

$$\mathbf{h}_u^{(t)} = \text{UPDATE}_{\text{concat}}(\mathbf{h}_u, \mathbf{m}_{\mathcal{N}(u)}) = [\text{UPDATE}_{\text{base}}(\mathbf{h}_u, \mathbf{m}_{\mathcal{N}(u)}) \oplus \mathbf{h}_u^{(t-1)}]$$

5. **Pooling (Readout):** Pool the final node messages $\mathbf{h}_u^{(T)}$ to get the embedding of the entire graph $\mathbf{h}_{\mathcal{G}}$. A simple way of doing this would be summing the node embeddings and dividing by the number of nodes:

$$\mathbf{h}_{\mathcal{G}} = \frac{1}{|\mathcal{V}|}\sum_{u\in\mathcal{V}}\mathbf{h}_u^{(T)}$$

4 Aggregation Step (Message Passing)

The aggregation step computes a neighborhood message vector $\mathbf{m}_{\mathcal{N}(u)}$ for each node u by aggregating the feature vectors $\mathbf{h}_v$ of its neighbors $v \in \mathcal{N}(u)$.

4.1 Message Vector Definition

The message vector $\mathbf{m}_{\mathcal{N}(u)}$ is defined as the average of the neighbor embeddings:

$$\mathbf{m}_{\mathcal{N}(u)} = \frac{\sum_{v\in\mathcal{N}(u)}\mathbf{h}_v}{|\mathcal{N}(u)|}$$

(*Equation 1: Aggregation*)

4.2 Parallel Implementation Details

During the aggregation phase, a new computational kernel is launched. The process is parallelized at the level of the neighbors:

- **Neighbor-wise Parallelism:** Each computational thread is responsible for handling the contribution of a single neighbor v to the message vector of a target node u. For instance, one thread handles the pair (u, v_1), another handles (u, v_2), and so on, where $v_i \in \mathcal{N}(u)$.
- **Accumulation:** Each thread loads the embedding $\mathbf{h}_v$, scales it by the inverse of the number of neighbors, $1/|\mathcal{N}(u)|$ (though in this specific implementation, it loads $\mathbf{h}_v$ and adds it to an accumulator of u before the final division), and adds it to the accumulator corresponding to the target node u.
- **Initialization:** The accumulator for node u must be reset every time the aggregation step starts anew to ensure a fresh sum.
- **Independence:** Since no thread's computation for u depends on another thread's computation for the same u until the accumulation is complete, the threads can execute independently. Thread blocks and warps simply represent a grouping of these independent threads.

Once all contributions are accumulated, the aggregation step for the current iteration is complete. Another way to think of this is to say, each thread handles an edge.

5 Update Step

After the message vector $\mathbf{m}_{\mathcal{N}(u)}$ is computed, a new kernel is launched to perform the update step for each node.

5.1 Node Embedding Update

The new embedding $\mathbf{h}_u^{\text{new}}$ is computed by combining the node's current embedding $\mathbf{h}_u$ and the aggregated message $\mathbf{m}_{\mathcal{N}(u)}$ through a non-linear transformation:

5.2 Base Update Function

The base update function combines the node's current embedding $\mathbf{h}_u$ and the aggregated message $\mathbf{m}_{\mathcal{N}(u)}$:

$$\text{UPDATE}_{\text{base}}(\mathbf{h}_u, \mathbf{m}_{\mathcal{N}(u)}) = \sigma\left(\mathbf{W}_{\text{self}}\mathbf{h}_u + \mathbf{W}_{\text{neigh}}\mathbf{m}_{\mathcal{N}(u)}\right)$$

(*Equation 2a: Base Update*)

5.3 Concatenation Update Function

The final embedding is produced by concatenating the result of the base update with the node's self-embedding $\mathbf{h}_u$, where $\oplus$ denotes concatenation, and the surrounding brackets indicate the resulting vector:

$$\text{UPDATE}_{\text{concat}}(\mathbf{h}_u, \mathbf{m}_{\mathcal{N}(u)}) = [\text{UPDATE}_{\text{base}}(\mathbf{h}_u, \mathbf{m}_{\mathcal{N}(u)}) \oplus \mathbf{h}_u]$$

(*Equation 2b: Concatenation Update*)

$$\mathbf{h}_u^{\text{new}} = \sigma\left(\mathbf{W} \cdot \left[\mathbf{h}_u \| \mathbf{m}_{\mathcal{N}(u)}\right]\right)$$

5.4 Parallel Implementation Details

- **Node-wise Parallelism:** This kernel uses one thread for each node u.
- **Computation:** Each thread loads the node's current embedding $\mathbf{h}_u$ and the pre-computed message vector $\mathbf{m}_{\mathcal{N}(u)}$.
- **Transformation:** The thread then applies the update function, which typically involves linear transformations ($\mathbf{W}_{\text{self}}$, $\mathbf{W}_{\text{neigh}}$) followed by an activation function (σ). The description suggests this could be implemented as a simple Multi-Layer Perceptron (MLP) or a concatenation of the transformed vectors. The result is the new embedding $\mathbf{h}_u^{\text{new}}$.

6 Iterative Process

The aggregation (Kernel 1) and update (Kernel 2) steps are distinct kernel launches (or grids) and constitute a single GNN layer. This process of aggregation and updation is repeated iteratively until the number of iterations (equivalent to the number of GNN layers) set by the user is reached.

7 Computational Bottleneck

There is no almost no data reuse when we go from one iteration of aggregation + update to the next we use an entirely new set of embeddings upon which we compute the next iterations embeddings. The process is not compute bound, only simple operations take place in this method of message passing. The process is however memory bound, due to constant access of data from the global memory for each thread.

Attribution Note:
Document Formatting: `https://gemini.google.com/share/73fadb4d126e`
Theoretical Discussion: `https://chatgpt.com/share/693a9e3b-3958-800e-a837-da1487353b70`
